

Introduction to Sensor Data Analysis and Anomaly Detection in Manufacturing

Habibor Rahman, Ph.D.

Assistant Professor

Mechanical Engineering

University of Massachusetts Dartmouth

mrahman15@umassd.edu

ACCOMPLISH program at UMass Dartmouth

Introduction to Sensor Data Analysis and Anomaly Detection in Manufacturing

This course module introduces students to the role of sensor data analytics in smart manufacturing. It connects basic theory to repeated hands-on examples in which students load data, visualize signals, extract features, detect anomalies, classify failure events, and interpret results from an engineering perspective.

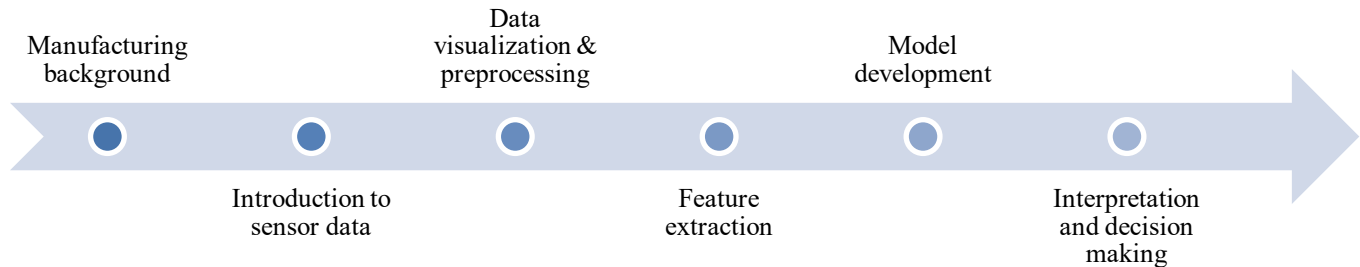


Figure 1: Module learning pathway – from manufacturing data to engineering decision-making

Intended course(s)	MNE 345 Design for Manufacturing; adaptable to lower-level engineering courses
Target audience	Undergraduate STEM students with limited prior exposure to Python, manufacturing analytics, or anomaly detection
Suggested duration	Two weeks (for a 1-week course plan, use only Chapters 1-7 and Case Study 1. The two-week plan requires chapters 8-11 and all three case studies)
Computing environment	Google Colab, Jupyter Notebook, or Anaconda Python

Contents

- Chapter 1. Module overview
- Chapter 2. Smart manufacturing, sensors, and process monitoring
- Chapter 3. From raw data to useful information
- Chapter 4. Types of data analytics in manufacturing
- Chapter 5. Classification, regression, and anomaly detection
- Chapter 6. Case study 1: equipment energy consumption anomaly detection
- Chapter 7. Signal segmentation and feature extraction
- Chapter 8. Case study 2: predictive maintenance classification
- Chapter 9. Case study 3: manufacturing process signal feature extraction
- Chapter 10. Model evaluation and engineering interpretation
- Chapter 11. Security-aware process monitoring
- Chapter 12. Student deliverables and grading rubric
- Chapter 13. Instructor guide
- Appendix A. Open data, code, and tutorial resources
- Appendix B. Code snippet library
- Appendix C. Glossary of key terms

1 Module Overview

Learning focus	Details
After this chapter, students should be able to	▪ Explain why computing and data analytics are important in modern manufacturing systems. ▪ Understand the complete learning pathway used in this module. ▪ Identify hands-on case studies and how they connect to theory.
Key terms	Smart manufacturing, sensor data, time series, feature, anomaly, classification, engineering interpretation
Connection to case studies	The same analytics ideas are revisited through three cases: equipment energy anomaly detection, predictive maintenance classification, and feature extraction from CNC process signal.

This module is designed for undergraduate students who may be new to programming and manufacturing analytics. Its primary objective is to introduce a concept, connect the concept to a small data example, and then ask students to perform guided tasks. Students, therefore, encounter theory as something they can use, not only as material to memorize.

The module is intentionally organized as a tutorial. Each chapter contains short theory, plain-language explanations, connections to manufacturing practice, and a hands-on activity. The three recurring case studies allow students to see how similar computational thinking applies across multiple manufacturing and engineering data problems.

Case study	Data type	Primary analytics task	Theory connected to the case	Student output
Case Study 1: Equipment energy consumption	Time-series data	Detect unusual energy-use behavior	Visualization, descriptive analytics, rolling statistics, thresholding, anomaly detection	Plots, anomaly flags, short engineering interpretation
Case Study 2: Predictive maintenance	Tabular machine-condition data	Classify machine failure risk	Classification, supervised learning, class imbalance, model evaluation	Classifier, confusion matrix, precision/recall discussion
Case Study 3: CNC process signal	High-frequency process power signal	Segment signal and extract features	Windowing, RMS, peak, energy, unsupervised anomaly detection	Feature table, anomaly model, engineering explanation

1.1 Learning objectives

- Interpret sensor data as evidence of physical behavior in a manufacturing system.
- Distinguish between descriptive, diagnostic, predictive, and prescriptive analytics.
- Differentiate classification, regression, and anomaly detection tasks.
- Load and visualize time-series and tabular manufacturing-related data using Python.

- Compute simple signal features, including mean, standard deviation, root-mean-square value, peak value, and energy proxy.
- Apply simple anomaly detection methods using thresholds, rolling statistics, and Isolation Forest.
- Train and evaluate a basic classification model for predictive maintenance.
- Explain model results using manufacturing reasoning, including false alarms, missed anomalies, and practical limitations.

1.2 Necessary files

File	Purpose
data/case_study_1_equipment_energy_power.csv	Power-consumption signal with labeled anomalies for Case Study 1
data/case_study_2_ai4i_like_predictive_maintenance_sample.csv	Predictive-maintenance classroom sample for Case Study 2
data/case_study_3_cnc_power_signal.csv	Simulated CNC power signal for Case Study 3
notebooks/01_energy_anomaly_detection_student.ipynb	Guided starter notebook for Case Study 1
notebooks/02_predictive_maintenance_classification_student.ipynb	Guided starter notebook for Case Study 2
notebooks/03_process_signal_feature_extraction_student.ipynb	Guided starter notebook for Case Study 3
notebooks/*_solution.ipynb	Completed or annotated solution notebooks

2 Smart Manufacturing, Sensors, and Process Monitoring

Learning focus	Details
After this chapter, students should be able to	▪ Describe why manufacturing systems use sensors for monitoring. ▪ Identify common sensor modalities used in machining and production systems. ▪ Explain the difference between offline and online inspection.
Key terms	Online monitoring, offline inspection, power sensor, force sensor, vibration, acoustic emission, process monitoring
Connection to case studies	Case Study 1 uses equipment power data; Case Study 3 uses process power data similar to signals that could be collected from CNC machines.

Manufacturing systems transform material into products through controlled physical processes. In machining, for example, the interaction between the tool and the workpiece generates forces, vibrations, heat, acoustic emissions, and increased power demand. Sensors can monitor these quantities. Sensor data provides a continuous or near-continuous record of how the process behaves over time.

Traditional inspection often occurs after a part is produced. This is valuable, but it may detect problems only after material, time, and labor have already been consumed. Online monitoring attempts to observe the process during production. It can support earlier detection of tool wear, chatter, overload, abnormal energy consumption, process drift, and potential cyber-physical manipulation.

Sensor or data source ¹	What it measures	What changes may suggest	Typical analytics use
Motor power/current	Electrical demand of the machine or spindle	Load change, tool wear, abnormal operation, idle energy waste	Energy monitoring, anomaly detection, process state identification
Force/torque	Mechanical load at the cutting interface	Tool wear, chatter, material inconsistency, overload	Tool condition monitoring and process stability
Vibration	Dynamic motion and oscillation	Chatter, imbalance, bearing issues, looseness	Frequency analysis, fault detection
Acoustic emission	High-frequency elastic waves	Crack initiation, tool-workpiece interaction, sudden events	Event detection and condition monitoring
Temperature/thermal imaging	Heat generation and thermal distribution	Cooling problem, excessive friction, abnormal heating	Thermal monitoring and defect risk assessment
Machine controller logs	Commanded operations, spindle speed, feed rate, alarms	Program changes, stoppages, abnormal sequence	Contextual interpretation and diagnostics

¹ Students do not need to know all sensor physics before using the data. They should first learn that each sensor is a measurement of some physical behavior, and analytical methods help convert those measurements into interpretable evidence about the machine and/or process condition.

Example: power signal interpretation

Suppose a machine normally consumes 20 kW during idle periods and 40 kW during production. If the signal suddenly jumps to 75 kW, several explanations are possible: a heavy cutting operation, tool wear, a jam, incorrect process settings, a sensor error, or an abnormal command. Data analytics can flag the event, but engineering judgment is needed to explain it. This distinction is important throughout the module: analytics can detect patterns, but engineers must interpret whether the pattern is meaningful, dangerous, expected, or caused by data quality issues.

3 Extracting Useful Information from Raw Data

Learning focus	Details
After this chapter, students should be able to	▪ Explain the basic steps from raw sensor data to engineering decisions. ▪ Recognize common data quality issues in time-series sensor data. ▪ Understand why preprocessing and visualization come before modeling.
Key terms	Raw data, timestamp, sampling rate, missing value, noise, outlier, preprocessing, visualization
Connection to case studies	Students first visualize raw power data in Case Study 1 before applying anomaly detection.

Raw sensor data rarely arrives in a form that directly answers an engineering question. The data may contain timestamps, missing readings, duplicated entries, noise, sensor drift, or values collected under different operating states. Before building a model, students should inspect the data and ask what each column means.

Step	Task	Questions students should ask	Example in this module
1	Understand the data source	What physical system generated the data?	An equipment meter records power consumption over time.
2	Inspect the table	What are the columns, units, and number of observations?	timestamp, power_kW, machine_state, anomaly_label
3	Visualize	What patterns are visible before modeling?	Daily energy cycles and unusual spikes.
4	Preprocess	Are there missing values, noise, or inconsistent timestamps?	Rolling statistics require enough previous observations.
5	Extract features	What numerical summaries describe behavior?	Mean, standard deviation, RMS, peak, energy proxy.
6	Model or detect	Which method answers the question?	Thresholding, Isolation Forest, or Random Forest.
7	Interpret	What does the result mean for the system?	A flagged period may indicate overload, idle waste, or a sensor fault.

Data quality issues

- Missing values: A sensor may fail to report at some timestamps.
- Noise: Readings may fluctuate around the true process value.
- Outliers: A value may be physically meaningful or may be a measurement error.
- Sampling-rate mismatch: Different sensors may report at different frequencies.

- Operating-state changes: Normal behavior during cutting may look abnormal during idle operation if the state is ignored.
- Label uncertainty: Failure or anomaly labels may be incomplete, delayed, or subjective.

Code snippet 3.1: First inspection of a sensor dataset

```
import pandas as pd

# Load a time-series dataset and inspect basic information
df = pd.read_csv('data/case_study_1_equipment_energy_power.csv',
parse_dates=['timestamp'])
print(df.head())
print(df.info())
print(df.isna().sum())
```


4 Types of Data Analytics in Manufacturing

Learning focus	Details
After this chapter, students should be able to	- Define descriptive, diagnostic, predictive, and prescriptive analytics. - Connect each analytics type to a manufacturing example. - Recognize that the same dataset can support different analytics questions.
Key terms	Descriptive analytics, diagnostic analytics, predictive analytics, prescriptive analytics
Connection to case studies	The same equipment power dataset can be used to ask what happened, why it happened, what may happen next, and what action should be taken.

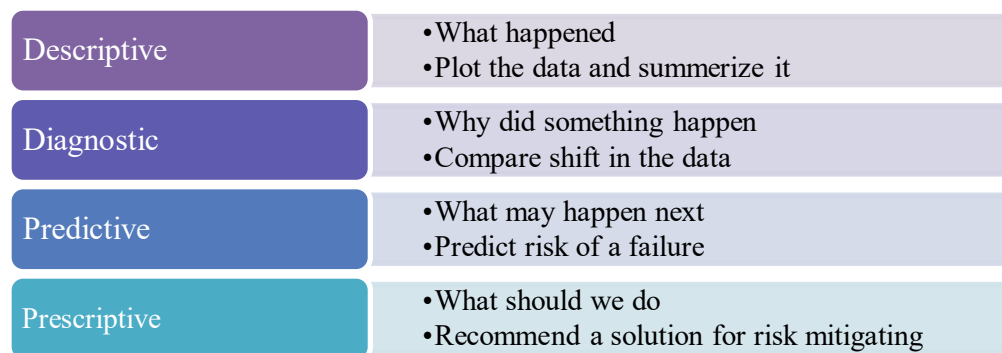


Figure 2: Types of data analytics on the same manufacturing domain problem

A common mistake is treating data analytics as a single activity. In manufacturing, analytics questions differ depending on the decision being supported. A plot that summarizes yesterday's power use is descriptive. A comparison of normal and abnormal shifts is diagnostic. A model that predicts the probability of machine failure is predictive. A rule that recommends maintenance or shutdown is prescriptive.

Analytics type	Plain-language question	Manufacturing example	Hands-on connection
Descriptive	What happened?	Plot the power signal and summarize the average, peak, and variability.	Case Study 1, Task 2
Diagnostic	Why did it happen?	Compare an abnormal power spike with the production schedule and machine state.	Case Study 1, Task 5
Predictive	What may happen next?	Predict whether the machine is likely to fail based on process variables.	Case Study 2
Prescriptive	What should we do?	Recommend inspection when the anomaly score exceeds a threshold and production risk is high.	Chapter 10 interpretation

Knowledge check

- a. If students compute the average power consumption for each day, which analytics type is being used?
- b. If students ask whether a spike occurred during production or during idle time, which analytics type is being used?
- c. If students train a model to predict whether a machine will fail, which analytics type is being used?
- d. If students recommend stopping a machine after multiple high-risk warnings, which analytics type is being used?

5 Classification, Regression, and Anomaly Detection

Learning focus	Details
After this chapter, students should be able to	▪ Distinguish among classification, regression, and anomaly detection. ▪ Select an appropriate analysis type for a manufacturing problem. ▪ Understand why labels matter in supervised learning.
Key terms	Classification, regression, anomaly detection, supervised learning, unsupervised learning, label
Connection to case studies	The energy case emphasizes anomaly detection; the predictive-maintenance case emphasizes classification; regression is introduced as an alternative question using the same types of data.

The same manufacturing dataset can support different problem formulations. The method should be selected after defining the question. Students should learn to ask: What is the input? What is the output? Is the output a class label, a continuous number, or an unknown unusual pattern?

Task type ²	Output	Example question	Example method	Module case
Classification	A category or class	Will this machine fail: yes or no?	Random Forest classifier	Case Study 2
Regression	A continuous value	What will the power consumption be in 1 hour?	Linear regression, random forest regression	Optional extension
Anomaly detection	A normal/abnormal flag or anomaly score	Is this power reading unusual compared with recent behavior?	Z-score, Isolation Forest	Case Studies 1 and 3
Clustering	Groups without labels	Do signals naturally separate into operating modes?	k-means clustering	Optional extension

5.1 Classification example

In predictive maintenance, a row may contain air temperature, process temperature, rotational speed, torque, and tool wear. If the target column is machine failure, the task is classification because the output is a label: failure or no failure.

² Classification requires examples with known labels. Anomaly detection can be useful when labels are unavailable, rare, or incomplete. In manufacturing, true failures are often rare, so anomaly detection is a practical first step.

5.2 Regression example

If the target is the expected power consumption during the next hour, the task is regression because the output is a continuous value such as 42.7 kW. Regression can also support anomaly detection by comparing predicted and observed values.

5.3 Anomaly detection example

If the goal is to find unusual behavior without knowing all possible failure modes, the task is anomaly detection. A high anomaly score does not automatically prove failure. It identifies a period that deserves attention.

6 Case Study 1: Equipment Energy Consumption Anomaly Detection

Learning focus	Details
After this chapter, students should be able to	- Load and visualize a time-series power dataset. - Detect anomalies using rolling statistics and z-score thresholding. - Explain detected events using engineering reasoning.
Key terms	Time series, rolling mean, rolling standard deviation, z-score, threshold, false alarm
Connection to case studies	This is the central undergraduate case. It is revisited when discussing analytics types, anomaly detection, evaluation, and interpretation.

This case study uses equipment power consumption as an intuitive entry point into anomaly detection. Power data are useful for teaching because students can reason about high loads, low loads, off-shift behavior, and sensor faults even before learning advanced manufacturing theory.

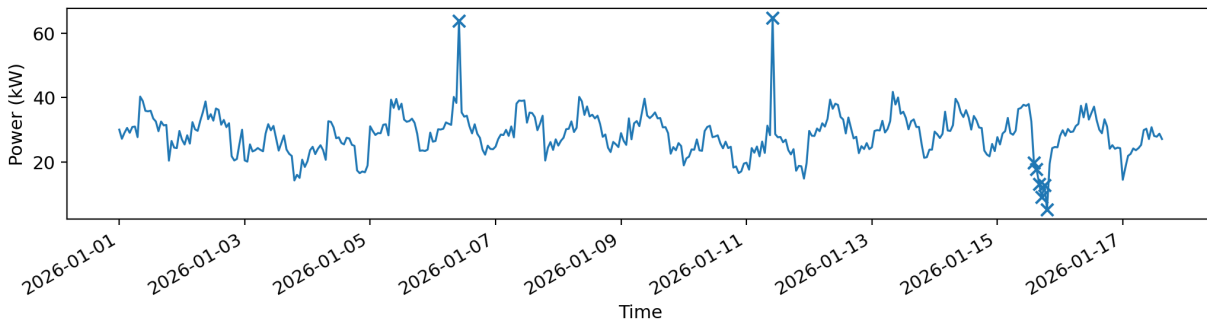


Figure 3: Sample power consumption signal with anomalies

- **Local classroom dataset included in this package:** [data/case_study_1_equipment_energy_power.csv](#) - use this file for stable classroom implementation
- **Open-data extension: Building Data Genome Project 2:** <https://github.com/buds-lab/building-data-genome-project-2> - large open energy-meter dataset for optional exploration
- **Open-data extension: Numenta Anomaly Benchmark:** <https://github.com/numenta/NAB> - labeled time-series anomaly datasets for optional exploration

6.1 Student tasks

Task	Student action	Purpose	Expected output
Task 1. Load the dataset	Read the CSV file using pandas and inspect the first rows.	Understand the columns and units.	Printed table preview and short data description.
Task 2. Plot the raw signal	Create a line plot of power_kW over time.	Develop visual intuition before modeling.	Time-series plot with observations about visible anomalies.

Task 3. Compute rolling statistics	Compute rolling mean and rolling standard deviation using a 24-hour window.	Estimate local normal behavior.	Columns for rolling_mean and rolling_std.
Task 4. Calculate z-scores	Compare each point with recent behavior.	Convert raw values to standardized deviations.	z_score column.
Task 5. Flag anomalies	Set a threshold such as $ z > 3$.	Build a simple rule-based detector.	detected_anomaly column and anomaly plot.
Task 6. Evaluate and interpret	Compare detected anomalies with provided labels and answer reflection questions.	Connect computation to engineering meaning.	Confusion matrix and short interpretation paragraph.

Code snippet 6.1: Load and inspect the power dataset

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data/case_study_1_equipment_energy_power.csv',
parse_dates=['timestamp'])
print(df.head())
print(df[['power_kW']].describe())
```

Code snippet 6.2: Plot the time-series signal

```
plt.figure(figsize=(10,4))
plt.plot(df['timestamp'], df['power_kW'])
plt.xlabel('Time')
plt.ylabel('Power (kW)')
plt.title('Equipment Power Consumption Over Time')
plt.show()
```

Code snippet 6.3: Rolling statistics and z-score

```
window = 24 # 24 hours of history

df['rolling_mean'] = df['power_kW'].rolling(window=window,
min_periods=window).mean()
df['rolling_std'] = df['power_kW'].rolling(window=window,
min_periods=window).std()
df['z_score'] = (df['power_kW'] - df['rolling_mean']) / df['rolling_std']

df[['timestamp', 'power_kW', 'rolling_mean', 'rolling_std', 'z_score']].head(30)
```

Code snippet 6.4: Threshold-based anomaly detection

```
threshold = 3.0

df['detected_anomaly'] = df['z_score'].abs() > threshold
```

```
print('Number of detected anomalies:', df['detected_anomaly'].sum())

plt.figure(figsize=(10,4))
plt.plot(df['timestamp'], df['power_kW'], label='Power')
anomalies = df[df['detected_anomaly']]
plt.scatter(anomalies['timestamp'], anomalies['power_kW'], marker='x', s=60,
label='Detected anomaly')
plt.xlabel('Time')
plt.ylabel('Power (kW)')
plt.title('Detected Anomalies in Equipment Power Consumption')
plt.legend()
plt.show()
```

6.2 Knowledge check

- Which detected events are sudden spikes? What are gradual changes?
- Could a high-power event be normal during a heavy production period? What additional information would you need?
- Could a low-power event indicate a machine stoppage, a sensor problem, or an off-shift period?
- What happens when the threshold changes from 3.0 to 2.0? Does the detector become more sensitive or less sensitive?
- Why is a simple threshold easier to explain than a black-box model? Why might it still be insufficient?

7 Signal Segmentation and Feature Extraction

Learning focus	Details
After this chapter, students should be able to	▪ Explain why raw time-series data are often divided into windows. ▪ Compute common time-domain features from each window. ▪ Connect features to physical process behavior.
Key terms	Windowing, segmentation, mean, standard deviation, RMS, peak, energy proxy, feature vector
Connection to case studies	Case Study 3 uses window-level features extracted from a CNC power signal.

Many manufacturing signals are too long to model point-by-point. A common approach is to divide the signal into short windows and compute features from each window. Each window becomes one row in a feature table. This feature table can then be used for classification, clustering, or anomaly detection.

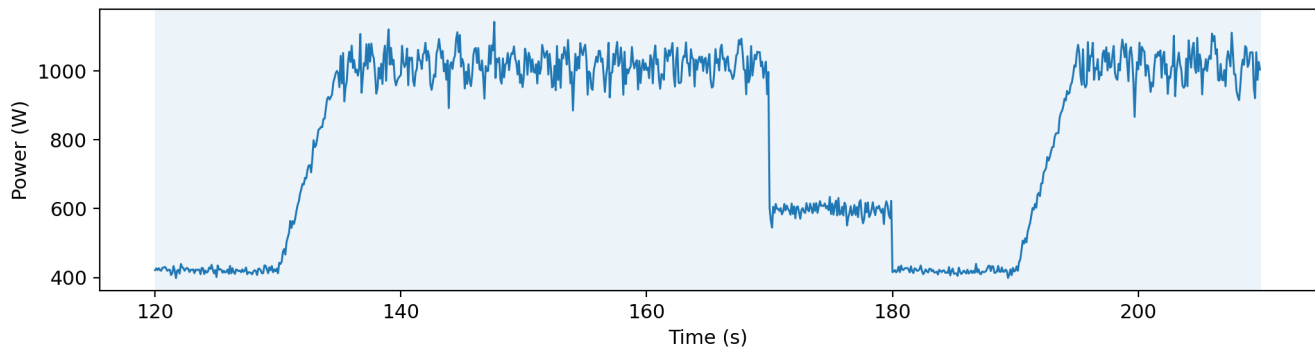


Figure 4: Signal segmentation example: splitting the power signal into short windows/timeframes

Feature	Formula or description	Physical interpretation of a power signal
Mean	Average value in the window	Typical load level during the window.
Standard deviation	Spread around the mean	Variability may increase during unstable operation or chatter.
RMS	Square root of the average squared value	Magnitude of signal energy; useful for load and vibration-like signals.
Peak	Maximum value in the window	Largest transient load or spike.
Energy proxy	Sum of signal values over time	Total accumulated power demand over the window.
Range	Maximum minus minimum	Amplitude of fluctuations.

Code snippet 7.1: Feature extraction from one signal window

```
import numpy as np

# Example: compute features from one window of signal values
x = segment['power_W'].values
features = {
    'mean_power': x.mean(),
    'std_power': x.std(),
    'rms_power': np.sqrt(np.mean(x**2)),
    'peak_power': x.max(),
    'energy_proxy': np.sum(x)
}
print(features)
```

Guided activity: feature meaning

- If mean power increases gradually over several windows, what manufacturing condition might be changing?
- If standard deviation increases while the mean remains similar, what types of abnormal behavior might be present?
- If peak power is high for only one window, is this necessarily a process failure? Explain.
- Why might feature extraction make an anomaly detection model easier to train than raw signal data?

8 Case Study 2: Predictive Maintenance Classification

Learning focus	Details
After this chapter, students should be able to	- Formulate a supervised classification problem. - Train a basic Random Forest classifier. - Interpret performance under class imbalance.
Key terms	Supervised learning, feature matrix, target label, Random Forest, class imbalance, SMOTE, precision, recall
Connection to case studies	This case complements anomaly detection by showing a labeled classification problem.

Predictive maintenance asks whether data can help anticipate machine degradation or failure. In a supervised setting, historical examples include both inputs and known labels. The model learns a relationship between features and a target label, then applies that relationship to new observations.

- Primary open dataset: UCI AI4I 2020 Predictive Maintenance Dataset:**
<https://archive.ics.uci.edu/ml/datasets/AI4I%2B2020%2BPredictive%2BMaintenance%2BDataset> - synthetic dataset reflecting predictive maintenance data encountered in industry
- Classroom fallback dataset included in this package:**
[data/case_study_2_ai4i_like_predictive_maintenance_sample.csv](#) - used when online access is unavailable
- scikit-learn RandomForestClassifier documentation:** <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html> - official code reference
- Imbalanced-learn SMOTE documentation:** https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html - optional extension for class imbalance

Task	Student action	Concept reinforced
Task 1. Load predictive-maintenance data	Use UCI AI4I or the included classroom sample.	Data source, features, target labels.
Task 2. Identify the target and features	Set Machine_failure as y and process variables as X.	Classification formulation.
Task 3. Inspect class balance	Count normal and failure cases.	Class imbalance.
Task 4. Train baseline classifier	Use Random Forest with a train/test split.	Supervised learning workflow.
Task 5. Evaluate model	Create a confusion matrix and classification report.	Precision, recall, F1-score.
Task 6. Interpret errors	Discuss false alarms and missed failures.	Engineering decision-making.

Code snippet 8.1: Define features and target for classification

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report

df = pd.read_csv('data/case_study_2_ai4i_like_predictive_maintenance_sample.csv')
features = ['Air_temperature_K', 'Process_temperature_K', 'Rotational_speed_rpm',
'Torque_Nm', 'Tool_wear_min']
X = df[features]
y = df['Machine_failure']
print(y.value_counts())
```

Code snippet 8.2: Train and evaluate a Random Forest classifier

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = RandomForestClassifier(n_estimators=100, random_state=42,
class_weight='balanced')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, zero_division=0))
```

Code snippet 8.3: Feature importance for interpretation

```
import pandas as pd
import matplotlib.pyplot as plt

importance = pd.Series(model.feature_importances_,
index=features).sort_values(ascending=False)
importance.plot(kind='bar')
plt.ylabel('Feature importance')
plt.title('Which variables influenced the classifier?')
plt.show()
```

Optional extension: SMOTE

Failure examples are often rare. This means a model may appear accurate simply by predicting the majority class. SMOTE is an optional method for generating synthetic examples of the minority class in the training set. It should be introduced carefully: resampling must be applied only to the training data, not to the test data.

Code snippet 8.4: Optional SMOTE pipeline

```
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline

pipeline = Pipeline(steps=[
```

```
    ('smote', SMOTE(random_state=42)),  
    ('rf', RandomForestClassifier(n_estimators=100, random_state=42))  
])  
  
pipeline.fit(X_train, y_train)  
y_pred_smote = pipeline.predict(X_test)  
print(classification_report(y_test, y_pred_smote, zero_division=0))
```

9 Case Study 3: Manufacturing Process Signal Feature Extraction

Learning focus	Details
After this chapter, students should be able to	▪ Segment a process signal into windows. ▪ Compute window-level features. ▪ Apply Isolation Forest to feature-based anomaly detection.
Key terms	CNC signal, window-level features, Isolation Forest, unsupervised anomaly detection
Connection to case studies	This case is closest to manufacturing process monitoring and connects directly to machining sensor analytics.

This case uses a simulated power signal for a CNC machine. The signal includes normal idle, ramp-up, cutting, and unloading behavior, along with abnormal intervals that represent unstable operation and increasing load. The objective is not to create a perfect detector; it is to help students understand how raw process signals become feature tables that can support analytics.

- **Local classroom dataset included in this package:** [data/case_study_3_cnc_power_signal.csv](#) - simulated CNC power signal
- **Open manufacturing dataset extension: NASA Milling Wear dataset:** <https://data.nasa.gov/dataset/milling-wear> - milling experiments with different speeds, feeds, depth of cut, and wear records
- **PyWavelets documentation:** <https://pywavelets.readthedocs.io/> - optional extension for wavelet-based signal decomposition
- **scikit-learn IsolationForest documentation:** <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html> - official anomaly detection model reference

Code snippet 9.1: Load and plot the CNC power signal

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('data/case_study_3_cnc_power_signal.csv')
plt.plot(df['time_s'], df['power_W'])
plt.xlabel('Time (s)')
plt.ylabel('Power (W)')
plt.title('CNC Power Signal')
plt.show()
```

Code snippet 9.2: Convert raw signal into a feature table

```
import numpy as np

window_seconds = 10
fs = 10
window_size = window_seconds * fs
rows = []
```

```

for start in range(0, len(df) - window_size + 1, window_size):
    segment = df.iloc[start:start + window_size]
    x = segment['power_W'].values
    rows.append({
        'start_time_s': segment['time_s'].iloc[0],
        'mean_power': x.mean(),
        'std_power': x.std(),
        'rms_power': np.sqrt(np.mean(x**2)),
        'peak_power': x.max(),
        'energy_proxy': np.sum(x) / fs,
        'anomaly_label': int(segment['anomaly_label'].max())
    })

features = pd.DataFrame(rows)
features.head()

```

Code snippet 9.3: Isolation Forest anomaly detection

```

from sklearn.ensemble import IsolationForest

feature_cols = ['mean_power', 'std_power', 'rms_power', 'peak_power',
                'energy_proxy']
model = IsolationForest(contamination=0.12, random_state=42)
features['iforest_pred'] = model.fit_predict(features[feature_cols])
features['detected_anomaly'] = (features['iforest_pred'] == -1).astype(int)
features[['start_time_s', 'detected_anomaly']].head()

```

Optional extension: wavelet decomposition

Wavelet decomposition can separate slower trends from sharper changes in a signal. This is useful when students are ready to see a more advanced signal-processing method. It is optional and should not be required for students with a limited programming background.

Code snippet 9.4: Optional discrete wavelet transform

```

# Optional: wavelet decomposition
# pip install PyWavelets
import pywt

x = df['power_W'].values[:1024]
cA, cD = pywt.dwt(x, 'db1')
print('Approximation length:', len(cA))
print('Detail length:', len(cD))

```

10 Model Evaluation and Engineering Interpretation

Learning focus	Details
After this chapter, students should be able to	- Read a confusion matrix in the context of anomaly detection. - Explain precision, recall, and F1-score using engineering consequences. - Discuss why accuracy alone can be misleading.
Key terms	Confusion matrix, precision, recall, F1-score, false alarm, missed anomaly, and accuracy.
Connection to case studies	Evaluation is applied to both Case Study 1 and Case Study 2.

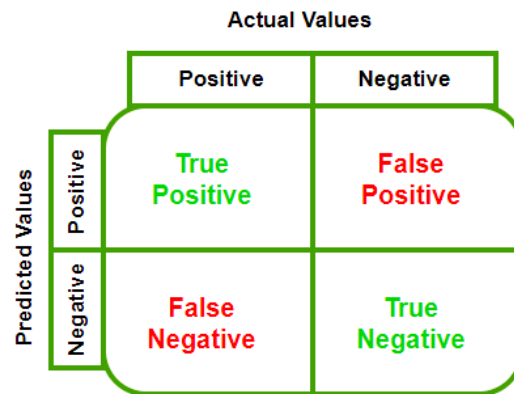


Figure 5: Confusion matrix for manufacturing anomaly detection

Model evaluation should not be treated as a purely mathematical exercise. In manufacturing, the costs of a false alarm and a missed anomaly are different. A false alarm may unnecessarily stop production. A missed anomaly may lead to defective parts, machine damage, safety risks, or continued energy waste. Students should learn to interpret metrics in the context of engineering decisions.

Metric	Meaning	Manufacturing interpretation
Accuracy	Fraction of all correct predictions	It can be misleading when anomalies are rare.
Precision	Of the events flagged as anomalies, how many were truly anomalous?	Low precision means many false alarms.
Recall	Of the true anomalies, how many did the model detect?	Low recall means many missed anomalies.
F1-score	Balance between precision and recall	Useful when both false alarms and missed anomalies matter.
Confusion matrix	Counts of correct and incorrect predictions	Helps identify the type of error being made.

Code snippet 10.1: Generic evaluation code

```
from sklearn.metrics import confusion_matrix, classification_report

print(confusion_matrix(y_true, y_pred))
print(classification_report(y_true, y_pred, zero_division=0))
```

Knowledge check

- The model detected ____ anomalies and missed ____ anomalies.
- The most concerning error type is ____ because in this manufacturing context it could lead to ____.
- The threshold/model setting should be adjusted toward higher recall if ____.
- The threshold/model setting should be adjusted toward higher precision if ____.
- Before deploying this detector, an engineer should validate it using ____.

11 Security-Aware Process Monitoring

Learning focus	Details
After this chapter, students should be able to	▪ Explain why anomaly detection is useful but not sufficient for cyber-physical manufacturing security. ▪ Describe how abnormal data patterns may arise from process faults, sensor faults, or malicious manipulation. ▪ Connect process monitoring to security-aware decision-making.
Key terms	Cyber-physical system, security-aware monitoring, process anomaly, data manipulation, attack detection
Connection to case studies	The same signals used for process monitoring can also provide evidence of suspicious or manipulated behavior.

The previous chapters treated anomalies mainly as process or equipment issues. In modern manufacturing, abnormal data may also be related to cyber-physical concerns. For example, a malicious modification to process parameters may change power, force, vibration, or temperature signatures. In contrast, an attacker who understands the monitoring system may attempt to hide changes so that common signal descriptors remain within normal limits.

For undergraduate learners, the goal is not to master cybersecurity theory. The goal is to understand that data analytics supports security-aware monitoring only when it is connected to physical process knowledge. A model can flag abnormal behavior, but engineers must still ask whether the pattern is caused by normal process variation, sensor fault, maintenance activity, operator action, or deliberate manipulation.

Observed pattern	Possible benign explanation	Possible concerning explanation	Follow-up evidence needed
Sudden high power spike	Heavy cut, startup load, temporary material hardness change	Unauthorized parameter change or jam	Controller log, process plan, operator note
Gradual increase in power	Tool wear or thermal drift	Subtle process manipulation designed to evade threshold detection	Tool inspection, feature trend, part quality
Unexpectedly low power	Idle period, shutdown, sensor dropout	Interrupted process or manipulated signal	Schedule, sensor health, machine state
Normal power but poor part quality	Inspection variation or material issue	Attack designed to preserve monitored signal while changing geometry	Part inspection, G-code comparison, additional sensors

Discussion activity

1. Why should a security-aware monitoring system use more than one sensor or feature?
2. How could an attacker attempt to evade a threshold-based power detector?
3. Why is domain knowledge important when interpreting anomaly alerts?
4. What additional data would help distinguish tool wear from data manipulation?

12 Student Deliverables and Grading Rubric

Students should submit a short technical report and completed notebook outputs. The emphasis should be on clear reasoning, reproducible analysis, and the connection between theory and engineering interpretation. The assignment should not reward code alone without explanation.

Deliverable	Description	Suggested length or format
Completed notebook	Run the required case study notebook and add brief markdown answers.	One notebook file or PDF export.
Figures	Include raw signal plot, anomaly plot, and at least one evaluation plot or table.	Embedded in notebook/report.
Feature table or summary	Show selected extracted features and explain their meaning.	Small table or paragraph.
Model/evaluation results	Report threshold results, confusion matrix, or classification report as applicable.	One table plus explanation.
Engineering interpretation	Explain what the detected anomalies or predictions mean in a manufacturing context.	300-600 words.
Reflection	Discuss limitations and what additional data would improve the analysis.	150-300 words.

12.1 Rubric

Criterion	Excellent	Satisfactory	Needs improvement	Points
Conceptual understanding	Clearly explains analytics type, data type, and manufacturing context.	Explains most concepts but with minor gaps.	Definitions are vague or disconnected from the case.	20
Code execution and reproducibility	Notebook runs and outputs are organized, labeled, and reproducible.	The notebook mostly runs with minor issues.	Code is incomplete, disorganized, or not reproducible.	20
Visualization and feature extraction	Plots and features are appropriate and interpreted correctly.	Basic plots/features are included with limited interpretation.	Plots/features are missing or not meaningful.	20
Modeling/evaluation	Applies an appropriate detection/classification method and evaluates results.	Method applied, but evaluation is limited.	Modeling or evaluation is incorrect or absent.	20

Engineering interpretation	Connects results to realistic manufacturing decisions and limitations.	Some interpretation is present but general.	Results are reported without engineering meaning.	20
----------------------------	--	---	---	----

12.2 Grading consideration

- For a one-week lower-level module, grade only Case Study 1 and reduce the report to 2-3 pages.
- For an upper-level course, require all three case studies and ask students to compare threshold-based, supervised, and unsupervised approaches.
- For team-based implementation, assign each team one case and require a short in-class presentation.
- For students with a limited coding background, provide partially completed notebooks and grade them more heavily on interpretation than on syntax.

13 Instructor Guide

The module can be delivered as lectures plus guided lab sessions.

Session	Before class	In-class focus	After-class assignment
Session 1	Assign Chapters 1-3. Ask students to install Anaconda or open Google Colab.	Motivation, sensor data, raw data inspection. Run first data-loading examples.	Students complete Case Study 1 Tasks 1-2.
Session 2	Review Chapter 4.	Types of analytics. Use the energy case to distinguish descriptive, diagnostic, predictive, and prescriptive questions.	Students answer analytics-type practice questions.
Session 3	Review Chapter 5.	Classification vs regression vs anomaly detection. Discuss labels and unlabeled data.	Students complete Case Study 1 Tasks 3-5.
Session 4	Review Chapters 7 and 9.	Signal segmentation and feature extraction. Demonstrate window-level features.	Students complete Case Study 3 or selected feature-extraction tasks.
Session 5	Review Chapter 8.	Predictive maintenance classification and model evaluation.	Students complete Case Study 2 or an optional extension.
Session 6	Review Chapters 10-11.	Engineering interpretation and security-aware process monitoring.	Students submit a notebook and report.

13.1 Anticipated student difficulties and potential responses

Difficulty	Likely cause	Instructor response
Students focus only on code syntax.	New programmers may treat the notebook as a coding exercise.	Ask students to explain each output in manufacturing language.
Students think every outlier is a failure.	Anomaly detection terminology can imply certainty.	Emphasize that anomaly means unusual, not automatically defective.
Students overvalue accuracy.	Accuracy is familiar and simple.	Use rare-anomaly examples to show why recall and precision matter.
Students struggle with feature meaning.	Features may seem abstract.	Connect each feature to load, variability, transient behavior, or accumulated energy.
Students cannot distinguish between classification and anomaly detection.	Both can produce normal/abnormal outputs.	Ask whether known labels were used to train the model.

13.2 Recommended simplification for lower-level courses

For 100- or 200-level courses, the instructor may use only Case Study 1. Students can load the data, plot the power signal, compute rolling statistics, change the anomaly threshold, and write a short explanation. This still achieves the ACCOMPLISH goal of exposing students to computing through a disciplinary context while keeping the mathematical and programming demands manageable.

13.3 Recommended extension for upper-level courses

For upper-level courses, require students to compare at least two methods: z-score thresholding and Isolation Forest for anomaly detection, or Random Forest classification with and without class balancing. Students should discuss why a method may work well on one dataset but not another.

Appendix A. Open Data, Code, and Tutorial Resources

The following resources are included to help students and instructors extend the module beyond the provided classroom datasets. Instructors should preview large datasets before assigning them, as download sizes and file structures can be challenging for new learners.

- **UCI AI4I 2020 Predictive Maintenance Dataset:** <https://archive.ics.uci.edu/ml/datasets/AI4I%2B2020%2BPredictive%2BMaintenance%2BDataset> - Synthetic predictive-maintenance dataset suitable for classification.
- **Building Data Genome Project 2:** <https://github.com/buds-lab/building-data-genome-project-2> - Large open energy-meter dataset; useful for energy analytics and anomaly exploration.
- **NASA Milling Wear Dataset:** <https://data.nasa.gov/dataset/milling-wear> - Milling-machine experiments with speeds, feeds, depth of cut, and wear records.
- **Numenta Anomaly Benchmark:** <https://github.com/numenta/NAB> - Labeled time-series anomaly benchmark with real-world and artificial signals.
- **Scikit-learn Anomaly Detection Guide:** https://scikit-learn.org/stable/modules/outlier_detection.html - Official scikit-learn guide to outlier and novelty detection.
- **Scikit-learn IsolationForest:** <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html> - Official documentation for Isolation Forest.
- **Scikit-learn RandomForestClassifier:** <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html> - Official documentation for Random Forest classification.
- **Imbalanced-learn SMOTE:** https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html - Official documentation for SMOTE oversampling.
- **PyWavelets:** <https://pywavelets.readthedocs.io/> - Python wavelet-transform library for optional signal-processing extension.
- **Pandas:** <https://pandas.pydata.org/docs/> - Data loading and manipulation.
- **Matplotlib:** <https://matplotlib.org/stable/users/index.html> - Plotting and visualization.
- **Jupyter:** <https://jupyter.org/> - Notebook environment for interactive computing.

Appendix B. Code Snippet Library

This appendix collects reusable code patterns from the module. Students may copy and adapt these snippets, but they should also explain what each code block does.

B.1 Load a dataset

```
# Generic pattern: load a CSV file
import pandas as pd

df = pd.read_csv('path_to_file.csv')
df.head()
```

B.2 Inspect a dataset

```
# Generic pattern: inspect a dataset
print(df.shape)
print(df.columns)
print(df.info())
print(df.describe())
```

B.3 Plot a time series

```
# Generic pattern: plot a time series
import matplotlib.pyplot as plt

plt.plot(df['timestamp'], df['value'])
plt.xlabel('Time')
plt.ylabel('Value')
plt.title('Time-Series Signal')
plt.show()
```

B.4 Rolling z-score detector

```
# Generic pattern: rolling z-score anomaly detection
window = 24
df['rolling_mean'] = df['value'].rolling(window).mean()
df['rolling_std'] = df['value'].rolling(window).std()
df['z_score'] = (df['value'] - df['rolling_mean']) / df['rolling_std']
df['anomaly'] = df['z_score'].abs() > 3
```

B.5 Classification model

```
# Generic pattern: train/test classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
                                                    random_state=42, stratify=y)
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

Appendix C. Glossary of Key Terms

Term	Definition
Anomaly	A data pattern that differs from expected or normal behavior. It may indicate a fault, an unusual operating condition, a sensor issue, or another event.
Classification	A supervised learning task in which the output is a category or class label.

Confusion matrix	A table that counts true positives, false positives, true negatives, and false negatives.
Feature	A numerical value computed from raw data and used as input to an analytics model.
Regression	A supervised learning task in which the output is a continuous numerical value.
RMS	Root-mean-square: a measure of signal magnitude.
Rolling mean	The average value is computed over a moving window of recent observations.
Sensor data	Measurements collected from a physical process, machine, or environment.
Supervised learning	A learning setup in which labeled examples are available.
Unsupervised learning	A learning setup in which labels are not provided.
Windowing	The process of dividing a time-series signal into shorter segments for analysis.